

A REPORT

ON

FORECASTING AUTOMATION TOOL

BY

Name of the Student: Shwethaa Rajagopalan

ID No.: 2024TM93028

AT

JP Morgan Chase

Devarabeesanahalli, Bangalore

Karnataka – 560103



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

March 2026

A REPORT

ON

FORECASTING AUTOMATION TOOL

BY

Name of Student	ID No.	Discipline
Shwethaa Rajagopalan	2024TM93028	M.Tech Software Engineering

Prepared in partial fulfilment of the

WILP Dissertation Course No. S2-25_SEZG628T

AT

JP Morgan Chase

Devarabeesanahalli, Bangalore – 560103



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

March 2026

Acknowledgements

I would like to express my sincere gratitude to the management and leadership of JP Morgan Chase for providing the opportunity and infrastructure to carry out this dissertation work in a real-world production environment.

I am deeply indebted to my Supervisor, Srirama Santosh Sreepada, JP Morgan Chase, Bangalore, for his invaluable guidance, patient mentorship, and continuous encouragement throughout the duration of this project. His deep expertise in financial data engineering and distributed systems shaped the scope, direction, and quality of this work in ways that cannot be overstated.

I thank my Faculty Mentor, Prof. Ashish Narang, at BITS Pilani, for academic oversight and timely guidance in aligning the project with the program requirements.

I extend my sincere thanks to the Financial Planning & Analysis (FP&A) technology team at JP Morgan Chase for their domain insights, constructive feedback, and collaboration during the design and implementation phases of the pipeline.

Finally, I thank my family and colleagues for their unwavering support and understanding throughout this challenging but rewarding academic journey.

Shwethaa Rajagopalan

2024TM93028

March 2026

**BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
(RAJASTHAN)**

WILP Division

Organization: JP Morgan Chase
Location: Devarabeesanahalli, Bangalore – 560103
Duration: January 2026 – May 2026
Date of Start: January 2026
Date of Submission: May 2026

Title of the Project:

Forecasting Automation Tool

ID No. / Name of the Student:

2024TM93028 / Shwethaa Rajagopalan

Name and Designation of Supervisor:

Srirama Santosh Sreepada, Vice President, JP Morgan Chase, Bangalore

Name and Designation of Additional Examiner:

Ketan Kumar Joshi, Vice President, JP Morgan Chase, Mumbai

Name of Faculty Mentor:

Ashish Narang

Key Words:

Data Mesh, Databricks, PySpark, SNS Triggers, NII Forecasting, Medallion Architecture,
ETL Modernisation, Event-Driven Pipeline, Financial Forecasting, Delta Lake

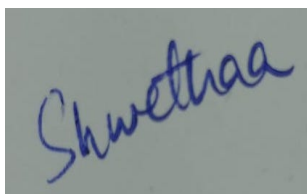
Project Areas:

Data Engineering, Financial Technology, Distributed Systems, Cloud Computing

Abstract:

The availability of quality data for timely decision-making has been a persistent challenge in financial institutions. Prior to this project, Net Interest Income (NII) forecasting at JP Morgan Chase was performed manually using Microsoft Excel workbooks, with business analysts fetching data from siloed, system-specific data marts populated by nightly batch jobs. This manual approach was constrained to higher levels of the business hierarchy, preventing granular forecasting at the cost-centre level and making it impossible to push top-level forecasts down to lower organisational nodes. Forecasting variables and assumptions were encoded directly in spreadsheet formulas, making scenario changes labour-intensive and error-prone. This project presents the Forecasting Automation Tool, a modernised NII forecasting system built at JP Morgan Chase using a Data Mesh architecture on the Databricks Lakehouse platform. The solution consumes data directly from authoritative data products through SNS-triggered, event-driven pipelines using PySpark. A ten-step computation pipeline progressively enriches data through configuration setup, actuals enrichment, forecast and launchpoint processing, yield delta computation, and NII staging at Business Unit and cost-centre granularity. The system eliminates manual data fetching and spreadsheet-based computation, reduces reconciliation effort by approximately 65%, delivers forecast output two business days earlier, and for the first time enables consistent NII allocation down to cost-centre level across the full business hierarchy. The configuration-driven design supports multiple concurrent forecast scenarios without code changes.

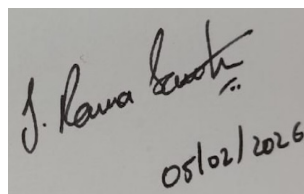
Signature of Student:



Shwethaa Rajagopalan

Date: 12/05/2026

Signature of Supervisor:



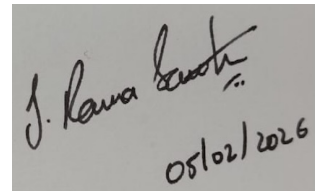
Srirama Santosh Sreepada

Date: 12/05/2026

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Dissertation entitled **Forecasting Automation Tool** and submitted by **Ms Shwethaa Rajagopalan** having ID No. **2024TM93028** in partial fulfillment of the requirements of Dissertation, embodies the work done by her under my supervision.



Signature of the Supervisor

Srirama Santosh Sreepada

Vice President

JP Morgan Chase

Place: Bangalore

Date: 12/05/2026

Contents

Acknowledgements	iii
Abstract Sheet	iv
1 Introduction	1
1.1 Background	1
1.2 Purpose and Motivation	3
1.3 Objectives	5
1.4 Scope and Limitations	6
1.5 Methodology Adopted	6
2 Review of Literature	7
2.1 Data Architecture Evolution	7
2.2 Event-Driven Processing and NII Modelling	8
2.3 Identified Gaps	8
3 Problem Statement	10
3.1 Problem Definition	10
3.2 Formal Problem Specification	12
3.2.1 Core NII Identity	12
3.3 Assumptions and Constraints	14
3.4 Relevance and Impact	14
4 Methodology Used	16

4.1	System Architecture	16
4.1.1	Reference Data Layer	16
4.1.2	Firm-Wide Data Warehouse	16
4.1.3	Forecast Data Product	16
4.1.4	Transformation Layer	18
4.1.5	Computation Layer	18
4.2	Technology Stack	18
4.3	Design Patterns	20
4.4	Configuration Management	21
4.4.1	Macroeconomic Variables	21
4.4.2	Run Scope and Hierarchy Definition	22
4.4.3	Yield Index Mapping	22
4.4.4	Launchpoint Window Definitions	23
4.4.5	Proxy Source Mappings	23
4.4.6	Inflation Adjustment Structures	23
4.4.7	Custom Yield Overrides	24
4.4.8	Launchpoint Override Rules	24
4.4.9	Forecast Substitution Rules	24
4.4.10	Launchpoint Period Yield Overrides	25
4.5	Pipeline Orchestration	26
4.6	Detailed Description of Pipeline Steps	27
4.6.1	Step 1: Config Update	27
4.6.2	Step 2: Actuals Enrichment	27
4.6.3	Step 3: Forecast Enrichment	28
4.6.4	Step 4: Launchpoint Enrichment	28
4.6.5	Step 5: Calculation Lookup	28
4.6.6	Step 6: Proxy Enrichment	28

4.6.7	Step 7: Inflation Adjustment Lookup (Parallel Branch)	29
4.6.8	Step 8: Yield Delta	29
4.6.9	Step 9: NII Stage	29
4.6.10	Step 10: NII Stage CC	30
4.7	Error Handling and Resilience	30
5	Results and Discussion	31
5.1	Evaluation Framework	31
5.2	Correctness Results	31
5.3	Performance Results	32
5.4	Operational Reliability Results	33
5.5	Business Impact Results	34
5.6	Discussion	36
5.6.1	Strengths of the Approach	36
5.6.2	Identified Limitations	37
5.6.3	Comparison with Commercial Tools	37
6	Conclusions and Recommendations	38
6.1	Conclusions	38
6.2	Recommendations	39
6.2.1	Automated Late Revision Handling	40
6.2.2	Real-Time Scenario Simulation	40
6.2.3	Machine Learning Enhanced Forecasting	40
6.2.4	Automated Data Quality Monitoring	40
A	List of Abbreviations	41
	References	41
	Glossary	43

List of Figures

4.1	End-to-End Forecasting Automation Tool Architecture	17
4.2	Three-Layer Summary: Reference Data, Firm-Wide Warehouse, and Forecast Data Product	19
4.3	Ten-Step NII Computation Pipeline DAG	27

List of Tables

4.1	Technology Stack	19
4.2	Configuration Parameter Summary	26
5.1	Correctness Metrics and Results	31
5.2	Performance Metrics and Results	33
5.3	Operational Reliability Metrics and Results	34
5.4	Business Impact Metrics and Results	34
A.1	List of Abbreviations and Acronyms	41

1. Introduction

1.1 Background

Financial institutions operate in highly data-intensive environments where timely and accurate forecasts drive critical business decisions: capital allocation, interest rate risk management, regulatory reporting, and strategic planning. Generating reliable Net Interest Income (NII) forecasts across multiple business units, business segment groups, and cost centres is central to the firm’s monthly financial planning cycle.

Prior to this project, NII forecasting at the firm was conducted primarily through manual, spreadsheet-driven processes. Financial analysts maintained complex Excel workbooks containing hard-coded formulas and static assumptions to project balance and income trajectories. This approach was deeply limited in multiple structural ways. First, forecasting was restricted to a single currency: all NII forecasts were produced exclusively in USD, preventing the firm from conducting multi-currency scenario analysis or forecasting for foreign operations with the same analytical rigour applied to the domestic portfolio. Second, forecasting granularity was constrained: analysts could only produce forecasts at higher levels of the business hierarchy, typically at the Business Unit level, and lacked the tooling to propagate those forecasts downward to Business Segment Groups or individual cost centres. Conversely, lower-level groups within the hierarchy could not independently generate forecasts that rolled up consistently to firm-wide totals. Second, the variables and assumptions embedded in these spreadsheets were rigid. Changing a macroeconomic assumption, a yield mapping, or an inflation adjustment structure required manual edits across multiple workbooks, with no systematic versioning or governance trail.

The data underpinning these manual forecasts was equally fragmented. Each system maintained its own local data mart. Business users were required to manually extract data from each of these systems, reconcile differences across mart boundaries, and load the consolidated dataset into their spreadsheets before any computation could begin. This data sourcing step was driven by

nightly batch jobs that populated each system’s mart on a fixed schedule, meaning analysts had no visibility into intra-day data changes and were dependent on the previous night’s snapshot regardless of whether upstream revisions had occurred during the business day. Because each system maintained its own mart independently, the same underlying general ledger data existed in multiple copies simultaneously, each at a different state of currency. This duplication created a persistent risk that numbers presented by different teams would be inconsistent, and that incorrect or stale figures would flow through to downstream end systems and ultimately into the hands of senior decision-makers without any mechanism to detect the divergence.

The transition to a Data Mesh architecture resolves these fragmentation problems at their root. Rather than each system maintaining a separate mart, all data is published through a single, governed data product layer and sourced from that common location by every consumer. This eliminates duplication: there is only one version of each balance, classification, or reference record at any point in time, sourced directly from the general ledger. The general ledger data underpinning the mesh is refreshed every four hours, a significant improvement over the previous once-daily nightly batch, meaning that analysts and the computation pipeline alike operate on data that is at most four hours old rather than potentially a full business day out of date. Data is also now available at a finer level of granularity than the nightly mart extracts provided, with account-level detail preserved through the mesh rather than being pre-aggregated during the batch load. This richer granularity enables more precise computation at the individual account and cost-centre level. Traceability is a further benefit: because every data element in the pipeline can be traced back to its originating data product and the specific refresh cycle from which it was consumed, the system provides a complete and auditable lineage from source general ledger entry to final NII output. This lineage is a prerequisite for data governance adherence in a regulated environment, enabling both internal audit and external regulatory review to verify the provenance of every figure in the forecast.

Beyond these manual processes, financial data platforms at large banks have historically relied on a centralised Extract, Transform, Load (ETL) paradigm. In this model, data is physically copied from source systems — general ledgers, transaction systems, and reference databases — into

a centralised data warehouse. While this approach provided a consistent view of data, it introduced critical limitations that compounded the problems of the manual process.

Data latency is the first such limitation. Physical movement and transformation of data introduce delays ranging from several hours to an entire day, making near-real-time analytics infeasible. The second limitation is data duplication and staleness. Copied data diverges from the source over time, leading to reconciliation burdens and trust issues among consumers. Upstream dependency is a third concern: failures or delays in upstream systems propagate to all downstream consumers, creating systemic fragility in the pipeline. Additionally, scalability constraints emerge as data volumes grow, since ETL pipelines require proportionally more time and computational resources. Finally, reconciliation overhead is significant, as separate frameworks must be maintained to validate that copied data matches the source of truth.

These limitations become acutely problematic in a monthly forecasting cycle where data readiness on specific business days is a hard operational deadline. Delays in the availability of metadata, actuals, or configuration data — exacerbated by manual extraction and nightly job dependencies — can cascade into missed planning deadlines for financial executives.

1.2 Purpose and Motivation

The project is motivated by three converging drivers: one operational and two arising from industry trends in data engineering and cloud platform maturity.

The primary operational driver is the inadequacy of the existing manual forecasting process. Analysts spending significant time each month manually extracting data from siloed system marts, reconciling nightly snapshots, and computing forecasts in rigid Excel workbooks represents a structural bottleneck that introduces error risk, constrains forecast granularity, and prevents the business from responding quickly to changed assumptions. The inability to push forecasts to cost-centre level, or to aggregate independently produced lower-level forecasts into a consistent firm-wide view, is a direct consequence of the tooling limitations of the spreadsheet-based approach. Eliminating this manual burden and replacing it with a governed, automated pipeline is the central motivation for

this project.

The first industry trend is the emergence of Data Mesh as an architectural paradigm for decentralised, domain-oriented data ownership. The Data Mesh philosophy proposes treating data as a product owned by domain teams, exposing it through well-defined interfaces without requiring central replication. In the context of this project, the Data Mesh functions as a single authoritative source for all data consumed by the forecasting pipeline, removing the need for analysts to perform manual data sourcing steps entirely. Because all consumers draw from the same data product, the risk of different teams working from inconsistent copies of the same underlying data — a structural problem under the siloed mart model — is eliminated by design. The Data Mesh is refreshed from the general ledger every four hours, providing substantially more current data than the previous nightly batch cycle and enabling computation to proceed on data that reflects the state of the general ledger within the same business day. Data ingested through the mesh is available at account-level granularity, a finer resolution than the pre-aggregated mart extracts it replaces, supporting more precise NII computation at the cost-centre level. Every data element carries full lineage metadata — recording its source data product, the refresh cycle from which it was consumed, and the transformation applied — which directly supports traceability requirements and data governance adherence. This aligns directly with the problem at hand: rather than having analysts manually pull data from each system’s local mart, the pipeline can consume data directly from authoritative sources, triggered by events signalling data readiness, with the assurance that every consumer is operating on the same, single, non-duplicated version of the data.

The second trend is the maturity of cloud-based Data Lake platforms such as Databricks for unified analytics at enterprise scale. The Databricks Lakehouse platform combines the flexibility of a data lake with the reliability of a data warehouse, through ACID transactions, schema enforcement, and audit trails. Tools such as PySpark, Delta Lake, and Databricks Workflows provide the computational infrastructure for implementing the complex, multi-step financial computations that previously required manual spreadsheet intervention at enterprise scale.

1.3 Objectives

The primary objectives of this project are as follows.

1. To replace the manual, spreadsheet-based forecasting process with an automated, governed pipeline that eliminates analyst dependency on nightly data mart extracts, manual formula maintenance, and currency-specific constraints.
2. To enable multi-currency forecasting, allowing NII projections to be computed for USD and foreign currencies simultaneously through a unified computation framework, supporting global business unit analysis and foreign exchange exposure modelling.
3. To modernise data ingestion by replacing ETL-based data copying with event-driven, trigger-based consumption from authoritative data products hosted in the data lake.
4. To implement a multi-source data collection framework that aggregates metadata from the Reference Data Layer, actuals from the Firm-Wide Data Warehouse, and user-defined forecast snapshots into a unified computation layer.
5. To build a computation pipeline that progressively enriches data from configuration setup through to allocation at the lowest hierarchy level, enabling forecasts to be produced at cost-centre granularity and ensuring that top-level forecasts can be consistently disaggregated to lower business hierarchy levels.
6. To enable scenario-based forecasting by supporting multiple forecast configurations without manual intervention, overcoming the rigidity of hard-coded spreadsheet variables through externalised, governance-controlled configuration tables.
7. To reduce reconciliation overhead by consuming data directly from source systems, thereby eliminating the need for downstream reconciliation frameworks and the manual cross-system validation currently performed by analysts.
8. To improve data freshness and reliability, removing dependence on nightly batch jobs and targeting data readiness within the monthly planning cycle.

1.4 Scope and Limitations

The scope of this project encompasses the design and implementation of the end-to-end data pipeline on Databricks, development of trigger-based integration with the Reference Data Layer and Firm-Wide Data Warehouse using SNS, implementation of the ten computation steps as self-contained PySpark processor tasks, configuration management through a hierarchy of override tables, and cost-centre level NII allocation using historical ratios and Financial Executive node weights.

The project does not include front-end visualisation interfaces, real-time streaming beyond event-triggered batch processing, or modifications to upstream source systems.

1.5 Methodology Adopted

The solution adopts a layered, event-driven architecture that spans five layers: the Reference Data Layer, the Firm-Wide Data Warehouse, the Forecast Data Product, the Transformation Layer, and the Computation Layer. Amazon Simple Notification Service (SNS) triggers coordinate data readiness events between these layers. The computation pipeline is implemented as a directed acyclic graph (DAG) of ten processor tasks, each encapsulated as a PySpark class instantiated through a factory pattern. Configuration is externalised through a family of override tables, enabling full re-parameterisation of the forecast without code changes. A detailed description of the methodology is given in Chapter 4.

2. Review of Literature

2.1 Data Architecture Evolution

The relational data warehouse model, formalised by Inmon (1992) and extended through Kimball’s dimensional modelling (1996), has been the cornerstone of enterprise analytics for three decades. While ETL-based centralisation provides a governed view of data, it introduces latency, duplication, and brittleness — problems that are schedule-driven rather than event-responsive, making BD-8 deadline compliance inherently fragile.

The Data Lake concept, popularised around 2010, offered cost advantages but suffered from poor data quality and unmanaged “data swamps” (Dixon, 2010). The Lakehouse architecture, introduced by Armbrust et al. (2021), addresses these shortcomings by combining flexible lake storage with warehouse-grade reliability through Delta Lake, PySpark, and Databricks Workflows — the foundational platform for this project.

Dehghani (2019, 2022) proposed the Data Mesh paradigm, built on four principles: domain-oriented data ownership, treating data as a product, self-serve infrastructure, and federated governance. In this project, the Reference Data Layer and Firm-Wide Data Warehouse act as domain-owned Data Products, consumed through SNS triggers and Delta tables without replication. The Data Mesh provides a single authoritative source for all data consumed by the pipeline: rather than maintaining separate data marts that diverge over time, every consumer draws from the same data product, eliminating duplication and the associated risk of inconsistent or stale numbers flowing to downstream end systems. The mesh is refreshed from the general ledger every four hours, replacing a once-daily nightly batch cycle and ensuring that computation operates on data that is current within the same business day. Because data is available at account-level granularity through the mesh — rather than pre-aggregated at the mart level — the pipeline can perform computations at a finer resolution than was previously possible. Full lineage metadata is preserved for every data element, recording its source product and the specific refresh cycle from which it was consumed,

directly supporting traceability and data governance adherence in the regulated environment. Data within the pipeline is further organised using the Medallion Architecture — raw actuals as Bronze, enriched lookups as Silver, and final NII outputs as Gold (Garg, 2022).

2.2 Event-Driven Processing and NII Modelling

Event-Driven Architecture decouples producers from consumers through asynchronous event propagation (Fowler, 2017). This pipeline employs the event notification pattern: SNS publishes data-readiness signals that trigger downstream processing without producer awareness of consumers. AWS SNS is preferred over Apache Kafka (Kreps et al., 2011) given its lower operational overhead and native AWS integration, appropriate for monthly event-triggered rather than continuously streaming workloads.

Net Interest Income — the difference between interest earned on assets and interest paid on liabilities — is a core ALM metric subject to Basel III and IV regulatory requirements (BIS, 2019). Key modelling challenges include yield curve sensitivity, yield inflation adjustment across lag horizons, and launchpoint bias. Backward proration reconstructs monthly actuals from period-to-date figures, while scenario-based planning systems such as Anaplan and Oracle Hyperion apply parameterised overrides — the pattern replicated through the governance-controlled configuration parameter family in this project.

2.3 Identified Gaps

Four gaps in existing literature and practice are addressed by this project. First, the academic literature on financial forecasting automation focuses almost exclusively on model accuracy and algorithmic improvements; the equally significant challenge of replacing manual, spreadsheet-based forecasting workflows with governed, automated pipelines in a regulated institutional context remains largely undocumented. Second, practical Data Mesh implementations within regulated financial institutions remain sparsely documented, particularly in environments where compliance and data sovereignty constraints limit architectural freedom. Third, event-driven NII pipelines on

cloud lakehouse platforms are largely unexplored, as most published frameworks assume centralised warehouses or manual analyst-driven data collection steps. Finally, the challenge of extending forecast granularity from higher organisational levels down to cost-centre level through a consistent, hierarchy-aware allocation mechanism — superseding the ad-hoc spreadsheet methods that leave lower-level groups without authoritative NII estimates — has not been systematically addressed in the engineering literature.

3. Problem Statement

3.1 Problem Definition

The financial planning division executes a monthly forecasting cycle to project Net Interest Income across all business units, business segment groups, and cost centres. Prior to this project, this cycle was executed manually: financial analysts used Excel workbooks with hard-coded formulas to compute NII projections, sourcing their inputs by manually downloading data from system-specific data marts and reconciling differences across mart boundaries.

This manual process was structurally constrained in three fundamental ways. First, forecasting was only possible at higher levels of the business hierarchy. Analysts could forecast at the Business Unit level but had no systematic mechanism to disaggregate those forecasts to Business Segment Groups or cost centres, nor could lower-level groups produce independent forecasts that rolled up consistently to firm-wide totals. Second, the forecasting variables — yield mappings, inflation adjustment structures, launchpoint definitions, and macroeconomic assumptions — were embedded directly in spreadsheet formulas. Any change required coordinated manual edits across multiple workbooks, with significant risk of version drift and no governance trail. Third, each upstream system maintained its own local data mart, populated by nightly batch jobs. Analysts had no access to intra-day data; they depended entirely on the previous night’s snapshot, irrespective of whether upstream revisions had been made during the current business day.

Beyond these manual process constraints, the computation itself requires the integration of data from three heterogeneous source systems.

The first source is the Reference Data Layer, which provides metadata including dimension hierarchies, account-to-business-unit mappings, and product classifications. This data must be ready by BD-8. The second source is the Firm-Wide Data Warehouse, which provides account-level actual balance and income data, available at varying times within the month-end close cycle. The third source is user-defined forecast snapshots, where financial analysts take point-in-time snapshots

of forecast data to anchor forward projections.

The existing system requires these data sources to be physically extracted and loaded into a shared data mart before computation can begin. This approach suffers from six specific, measurable problems.

The first problem is BD-8 deadline risk. Late metadata delivery from the Reference Data Layer delays the entire pipeline because ETL jobs must complete before any computation can start, creating a hard sequential dependency. The second problem is actuals staleness: monthly actuals at the account level may be revised multiple times after initial availability, but the nightly ETL snapshot does not reflect these intra-cycle revisions. Third, there is a manual intervention dependency: pipeline failures due to upstream delays require manual monitoring, triage, and re-triggering, consuming significant operational capacity. Fourth, reconciliation complexity is high because copied data requires a separate reconciliation framework to verify alignment between the data mart and the source system, adding latency and audit overhead. Fifth, configuration rigidity means that changing forecast assumptions such as yield mappings or inflation adjustment structures requires re-running the entire ETL pipeline from the beginning, a constraint that directly mirrors the inflexibility of the legacy spreadsheet model. Sixth, hierarchy coverage is incomplete: the absence of automated cost-centre disaggregation logic means that the full BU-to-CC allocation must be performed manually, introducing inconsistency and preventing lower-level business groups from independently participating in the planning process. Seventh, data duplication across multiple system-specific marts creates a material risk that incorrect or mutually inconsistent figures will flow through to end systems and to business decision-makers: with no single authoritative version of each data element, there is no reliable mechanism to detect when the copy used for a particular computation has diverged from the general ledger source of truth. Eighth, the coarse granularity of mart extracts — pre-aggregated to the business unit level during the nightly batch — means that account-level detail is lost before computation begins, preventing analysis and allocation at the cost-centre level that the business requires.

3.2 Formal Problem Specification

Let $\mathcal{S} = \{S_1, S_2, S_3\}$ denote the three data sources, where S_1 is the Reference Data Layer, S_2 is the Firm-Wide Data Warehouse, and S_3 is the set of user forecast snapshots. Let $\mathcal{B}$ denote the set of all business units b , $\mathcal{T}$ the set of forecast periods t , and $\mathcal{K}$ the set of all cost centres c .

3.2.1 Core NII Identity

Net Interest Income (NII) for a specific balance segment is defined as the product of three primary components: the interest-bearing balance, the applicable yield, and the day count fraction used to convert annualized rates into the specific period. For a single business unit (b) over a given period (t), the base NII contribution is expressed as:

$$\text{NII}_{b,t} = B_{b,t} \times Y_{b,t} \times d_t \quad (3.1)$$

Where the components are defined as follows:

- **Forecast Balance ($B_{b,t}$):** This represents the launchpoint-adjusted average balance for a business unit in a given period. It is calculated by taking the raw average forecast balance ($\bar{B}_{b,t}$) and applying both a currency mix factor (α_b) and a scaling factor (σ_b) derived from the calculation lookup process.
- **Yield Delta ($Y_{b,t}$):** This is the forward yield applicable to the business unit for the period. It is constructed by merging the base yield index mapping, forward market rate curves, inflation adjustment corrections, and any custom yield overrides established within the governance-controlled configuration parameters.
 - The calculation incorporates the index rate ($y_{b,t}^{\text{base}}$), forward rate changes from the curve ($\Delta y_{b,t}^{\text{MEV}}$), and yield inflation sensitivity captured through an inflation adjustment coefficient ($\beta_{b,k}$) applied to prior market rate changes (Δr_{t-k}).
 - Finally, any net yield overrides ($\delta_{b,t}^{\text{ovr}}$) established in the system's configuration tables are applied.

- **Day Count Fraction (d_t):** This factor converts the annualized yield to the actual length of the period based on applicable conventions, such as Actual/360 or Actual/365. It is defined as the ratio of actual days in the period (n_t) to the day count basis (δ) specified in the system configuration.

Full NII Forecast at Business Unit Level

Substituting Equations (??) and (??) into (??), the complete NII forecast at the business unit and period level is:

$$\hat{N}_{b,t} = \bar{B}_{b,t} \times \alpha_b \times \sigma_b \times Y_{b,t} \times \frac{n_t}{\delta} \quad (3.2)$$

Cost-Centre Allocation

The BU-level forecast $\hat{N}_{b,t}$ is allocated to individual cost centres $c \in \mathcal{K}_b$ using historical cost-centre ratios and Financial Executive (FE) node weights:

$$\hat{N}_{b,t,c} = \hat{N}_{b,t} \times \frac{r_{c|b} \cdot w_{fe,c}}{\sum_{c' \in \mathcal{K}_b} r_{c'|b} \cdot w_{fe,c'}} \quad (3.3)$$

where $r_{c|b}$ is the historical NII ratio for cost centre c within business unit b , and $w_{fe,c}$ is the FE node weight assigned during the governance approval process.

Problem Statement

The formal problem is: given data sources $\mathcal{S}$, configuration tables $\mathcal{C}$, and the equations above, compute $\hat{N}_{b,t,c}$ for all $(b, t, c) \in \mathcal{B} \times \mathcal{T} \times \mathcal{K}$ and for all currencies in $\mathcal{L} = \{\text{USD}, \text{EUR}, \text{GBP}, \dots\}$ correctly, reliably, and within the BD-8 deadline, *without* requiring physical replication of data from any element of $\mathcal{S}$. The allocation in Equation (3.3) must satisfy the consistency constraint:

$$\sum_{c \in \mathcal{K}_b} \hat{N}_{b,t,c} = \hat{N}_{b,t} \quad \forall b \in \mathcal{B}, t \in \mathcal{T} \quad (3.4)$$

ensuring that cost-centre-level NII sums exactly to the BU-level forecast with no residual.

3.3 Assumptions and Constraints

The following assumptions and constraints govern the solution.

1. Data readiness signals are published by the Reference Data Layer and Firm-Wide Data Warehouse as SNS notifications when data is available; the pipeline must respond within a configurable service level agreement.
2. Source data products are treated as immutable at the point of consumption; the pipeline must handle late revisions through a re-trigger mechanism.
3. All configuration overrides are managed through a governance-controlled process; the pipeline must apply these without modifying source data.
4. NII must be computable at any level of the BU and BSG hierarchy through consistent roll-up and drill-down semantics.
5. The solution must run entirely within the existing Databricks infrastructure, subject to the firm's security and compliance constraints.

3.4 Relevance and Impact

The Forecasting Automation Tool addresses both the systemic data pipeline problems and the immediate operational burden imposed by the manual, spreadsheet-based forecasting process. By eliminating manual data extraction from siloed system marts and replacing nightly-job-dependent snapshots with event-triggered ingestion from a single authoritative Data Mesh, the pipeline removes the primary sources of data latency and analyst effort from the monthly cycle. Data latency is reduced from the six-to-eight hours typical of nightly ETL jobs to under thirty minutes from event trigger to output availability, with the underlying general ledger data refreshed into the Data Mesh every four hours.

A critical dimension of this improvement is data correctness. Under the previous siloed mart model, each system maintained its own copy of the data, creating a persistent risk that incorrect or

stale figures would be used in computation and would subsequently flow unchecked into downstream end systems and management reports. By consolidating all data consumption to a single Data Mesh layer sourced directly from the general ledger, the pipeline ensures that every computation draws from one consistent, non-duplicated version of each data element. Combined with full lineage tracking from the general ledger to the final NII output, this provides an auditable chain of custody that satisfies both internal data governance requirements and external regulatory expectations. Data is also ingested at account-level granularity through the mesh, a finer resolution than the pre-aggregated mart extracts it replaces, enabling more precise computation and allocation.

Critically, the system resolves two interrelated structural gaps that manual processes could not address. First, by automating cost-centre-level NII allocation through governance-controlled configuration tables, the pipeline enables financial executives at lower levels of the business hierarchy to receive systematic, auditable forecasts for the first time. Second, the solution eliminates the previous USD-only constraint by implementing a multi-currency computation framework. The pipeline now produces NII forecasts concurrently across USD and foreign currencies, enabling the firm to forecast and stress-test global operations with the same analytical rigour previously available only for the domestic portfolio. The configuration-driven design through the CFG table family replaces the rigid, hard-coded spreadsheet variables that previously made assumption changes laborious and error-prone: macroeconomic variables, currency-specific yield mappings, inflation adjustment structures, and launchpoint definitions can now be updated and applied across the full computation without code or formula changes. This directly supports regulatory stress testing scenarios under Basel IV compliance frameworks without requiring engineering intervention for each scenario change, and enables scenario analysis across multiple currencies and interest rate environments simultaneously.

4. Methodology Used

4.1 System Architecture

The Forecasting Automation Tool adopts a layered, event-driven architecture spanning five primary layers: the Reference Data Layer, the Firm-Wide Data Warehouse, the Forecast Data Product, the Transformation Layer, and the Computation Layer. The architecture is designed around the Data Mesh principle of a single authoritative data source: all layers consume data from the same governed data products rather than from independent copies, ensuring consistency across the pipeline and eliminating the duplication risk inherent in multi-mart architectures. Figure 4.1 illustrates the end-to-end data flow.

4.1.1 Reference Data Layer

The Reference Data Layer ensures that metadata — dimension hierarchies, product mappings, and account classifications — is available by BD-8 each month. When readiness is confirmed, it publishes SNS Trigger 1, which initiates the Transformation Logic for moving and transforming required metadata into the Forecast Data Product.

4.1.2 Firm-Wide Data Warehouse

The Firm-Wide Data Warehouse provides account-level actuals. Upon data availability, it publishes SNS Trigger 2, which initiates the Transformation Logic to summarise and consume actuals within the Forecast Data Product.

4.1.3 Forecast Data Product

The Forecast Data Product acts as the single authoritative data source for the entire computation pipeline. It integrates transformed metadata (Source 1) from the Reference Data Layer, summarised actuals (Source 2) from the Firm-Wide Data Warehouse, and user data snapshots (Source 3) taken

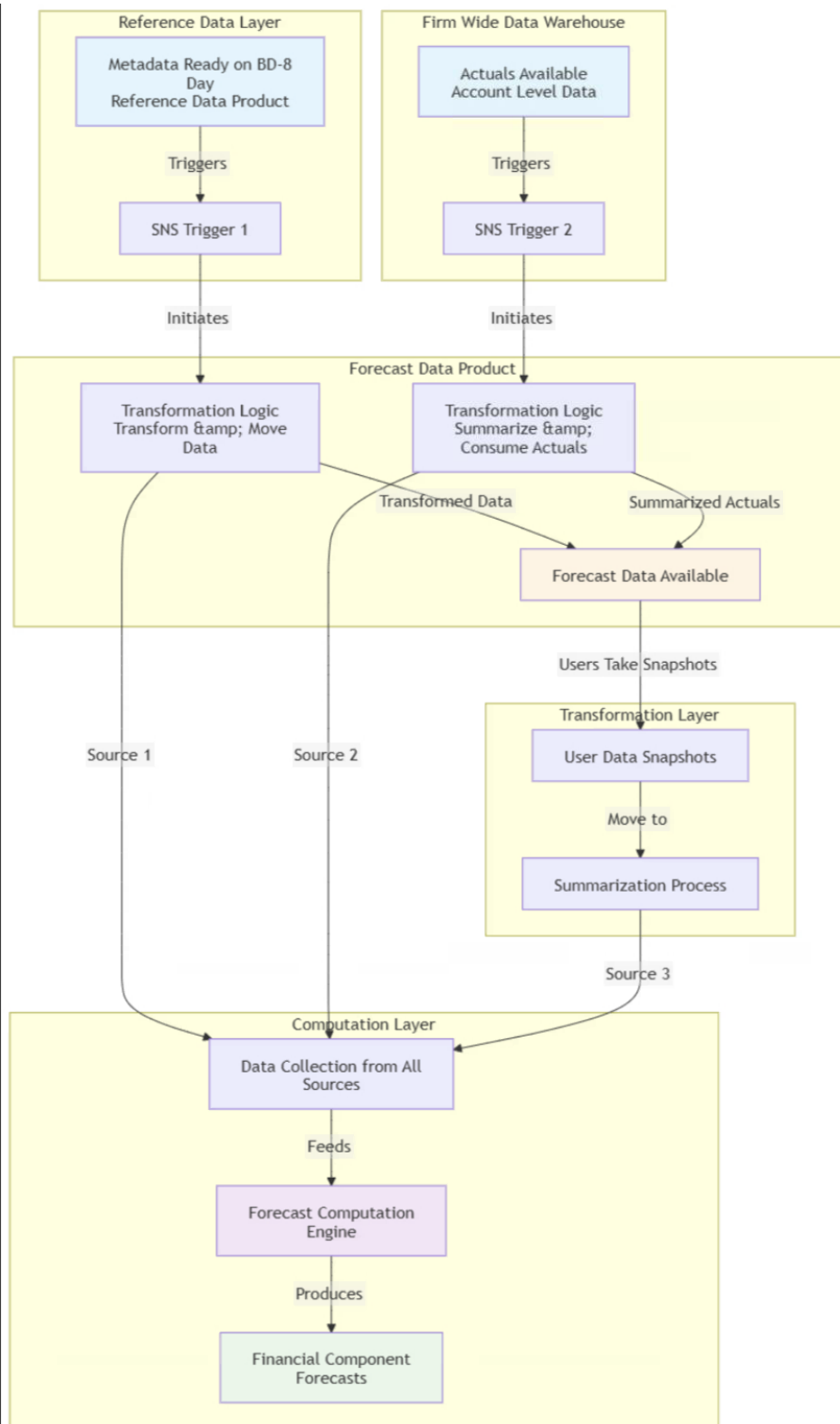


Figure 4.1: End-to-End Forecasting Automation Tool Architecture

by financial analysts and moved through the Transformation Layer for summarisation. Because all downstream computation steps consume exclusively from this single data product rather than from separate system-specific marts, there is no duplication of data within the pipeline: every step references the same version of each balance, classification, or reference record. This design also ensures that the risk of incorrect or inconsistent numbers flowing to end systems is mitigated structurally rather than through compensating reconciliation controls. The Forecast Data Product retains full lineage metadata for each data element, enabling complete traceability from the general ledger source through to the final NII output, and supporting data governance adherence and regulatory audit requirements. Data is made available through the product at account-level granularity — finer than the business-unit-level aggregations that nightly mart extracts provided — enabling more precise allocation computations in the downstream steps.

4.1.4 Transformation Layer

Financial analysts take point-in-time data snapshots of forecast data, which are moved to the Transformation Layer for summarisation before being passed to the Computation Layer.

4.1.5 Computation Layer

The Computation Layer aggregates all three sources, feeds them into the Forecast Computation Engine, and produces the Financial Component Forecasts — NII at the BU/BSG level and at cost-centre level.

4.2 Technology Stack

Table 4.1 summarises the technology stack used in the implementation.

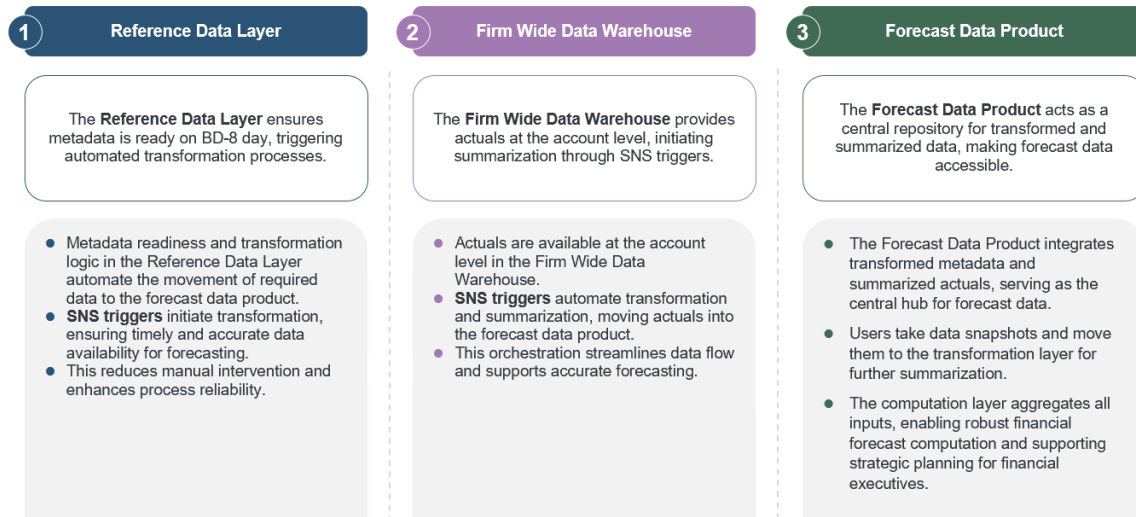


Figure 4.2: Three-Layer Summary: Reference Data, Firm-Wide Warehouse, and Forecast Data Product

Table 4.1: Technology Stack

Component	Technology		Role
Compute Engine	Apache	Spark	Distributed data processing for all pipeline steps
Data Platform	Databricks	Lakehouse	Unified environment for notebooks, workflows, Delta tables
Storage Format	Delta Lake		ACID transactions, time travel, schema enforcement
Event Bus	AWS SNS		Asynchronous trigger for pipeline initiation
Orchestration	Databricks Pipelines		Job scheduling, dependency management, retries
Configuration	YAML Task Files		Per-step configuration definitions
Language	Python 3 / PySpark		Business logic and transformations
Version Control	Bitbucket (Git)		Source code management
Issue Tracking	Jira		Task and sprint management
IDE	VS Code	with Copilot	Development with AI-assisted documentation

4.3 Design Patterns

The codebase is organised around four principal layers of concern: processor classes that encapsulate step-specific business logic, reader and writer abstractions that handle all data input and output, validation modules that enforce schema and data quality contracts, and a central task orchestration component that coordinates execution across all pipeline steps.

The pipeline implements a **factory pattern** for processor instantiation. Rather than the orchestration layer containing direct references to each step’s implementation, a central factory reads the task configuration at runtime and instantiates the appropriate processor class for that step. This decouples the orchestration logic from the business logic entirely: adding a new pipeline step requires only the creation of a new processor class and a corresponding configuration entry, with no changes to the orchestration layer itself. The factory pattern also simplifies testing, since each processor can be instantiated and exercised independently with mock configuration and data.

The **central task orchestration component** serves as the universal entry point for all pipeline step executions. It is responsible for reading the step-level configuration, invoking the factory to obtain the correct processor, supplying the processor with its data dependencies, and propagating any errors upward with structured diagnostic messages. Separating this orchestration concern from both the business logic and the data access layer means that changes to error handling, logging, or retry behaviour can be made in a single location without touching any step implementation.

All data access is mediated through a **reader-writer abstraction layer**. A pair of base classes defines the contracts for reading input data and writing output data, and concrete implementations of these contracts are provided for each supported storage format, including Delta Lake tables, Parquet files, CSV, JSON, and SparkSQL views. The business logic in each processor is written exclusively against the base contracts and is therefore entirely unaware of the physical storage format in use. This design enables the pipeline to switch data sources or output destinations — for example, to redirect a step’s output to a different storage layer during testing — without

modifying any business logic. It also provides a natural extension point for future storage formats.

A **validation module** is invoked at the boundary of each step before processing begins. It enforces schema contracts on incoming data, checks for the presence of required fields, and verifies that key data quality invariants hold before computation proceeds. When a validation check fails, the module raises a structured exception with a diagnostic message that identifies the specific field, constraint, and value that triggered the failure. This approach ensures that schema or data quality problems produce an immediate, actionable failure at the point of entry rather than a silent corruption that propagates through subsequent steps and becomes difficult to trace.

Together, these patterns produce a codebase in which each concern — orchestration, business logic, data access, and validation — is localised to a specific layer. Each step can be developed, tested, and modified in isolation. The architecture has proven particularly valuable during iterative deployment, where individual steps required adjustment without disrupting the rest of the pipeline.

4.4 Configuration Management

The pipeline is governed by a family of versioned override tables that collectively control every tunable parameter in the forecast computation. This design enables full re-parameterisation of any forecast run — including what-if scenario variants and version comparisons — without requiring any code changes. Each configuration table is managed through a controlled approval process, ensuring that every computation is traceable to a specific, auditable configuration state captured at the time the forecast instance is created. Business users can define multiple named configuration versions simultaneously and run the pipeline against each in parallel, making it straightforward to compare the NII impact of different macroeconomic assumptions, rate environments, or scope definitions side by side.

4.4.1 Macroeconomic Variables

The macroeconomic variable overrides, used in the Yield Delta step, represent the forward path of key macroeconomic indicators — such as GDP growth, unemployment rates, and central bank policy

rates — that drive the interest rate environment across tenors and currencies. By externalising these variables into a configuration table rather than hard-coding them, the pipeline can accommodate different macroeconomic scenarios — such as a rising-rate environment versus a recessionary flat-rate base case — simply by updating the configuration values without modifying any business logic. Macroeconomic variables are one of the two primary drivers of NII sensitivity to macro-economic assumptions, the other being the inflation adjustment structures.

4.4.2 Run Scope and Hierarchy Definition

The run scope configuration defines the top-level boundaries for each pipeline execution. It specifies which business units, business segment groups, and time horizons are in scope for a given run, along with the version identifiers that link the run to its governance-approved parameter set. The Config Update step reads this configuration first, using it to determine which downstream nodes need to be processed and which parameter versions apply. This ensures that a run scoped to a subset of business units — for example during a regional stress test — does not inadvertently pull parameters intended for a firm-wide planning cycle.

4.4.3 Yield Index Mapping

The yield index mapping configuration associates each combination of business unit, product type, and currency with a specific market rate index, such as a benchmark overnight rate, a prime lending rate, or a custom internal rate. This mapping is consumed in the Yield Delta step alongside the forward market rate curves to construct the applicable yield for each balance segment. Without this mapping, the yield delta computation would be unable to distinguish between, for example, a floating-rate consumer deposit and a fixed-rate commercial loan. The yield index mapping therefore determines which rate environment each product segment is exposed to, making it a critical driver of NII variance across scenarios.

4.4.4 Launchpoint Window Definitions

The launchpoint window configuration specifies, for each business unit, the start and end dates that define the period over which actual balances are observed to establish the forecast starting position. A wider window smooths out intra-month fluctuations, while a narrower window provides a more current but potentially noisier starting balance. The Launchpoint Enrichment step applies these windows to select and day-weight the relevant actual balance observations before passing them forward to the computation engine. Incorrect launchpoint windows are one of the most common sources of launch bias in financial forecasting, making this one of the most business-sensitive configuration tables in the pipeline.

4.4.5 Proxy Source Mappings

The proxy source mapping configuration defines the target-to-proxy relationships used in the Proxy Enrichment step. For business units or product segments with insufficient historical data to support a standalone forecast — typically newly created segments or those undergoing structural change — the configuration maps them to a proxy segment whose historical behaviour is considered representative. The table specifies whether the scaling between target and proxy is relative, expressed as a ratio, or fixed, expressed as an absolute adjustment. This configuration is essential for ensuring that no segment is silently excluded from the NII output due to data sparsity, which would cause the portfolio-level totals to be understated.

4.4.6 Inflation Adjustment Structures

The inflation adjustment configuration contains the inflation adjustment structures for each product and business unit. An inflation adjustment structure describes the degree to which a product's effective yield is adjusted for inflationary pressure over a defined horizon. The structure is parameterised from lag-0, representing the immediate adjustment applied in the current period, through lag-12, representing the cumulative adjustment distributed across the subsequent twelve months. These structures are loaded in the Inflation Adjustment Lookup step, which runs in parallel with

the main enrichment chain, and are subsequently merged in the Yield Delta step to modify forward yield curves for the inflation-driven yield dynamics of each product and business unit. This is the primary configuration driving NII sensitivity to inflationary rate scenarios, making its accurate calibration critical for regulatory stress testing.

4.4.7 Custom Yield Overrides

The custom yield override configuration provides a mechanism for applying bespoke yield values to specific business unit and product combinations that cannot be adequately represented by the standard yield index mapping combined with the forward market rate curves. This typically arises for structured products, certain intercompany funding arrangements, or segments where a negotiated rate is contractually fixed regardless of market movements. These overrides are applied in the Yield Delta step after the standard yield mapping, ensuring that standard segments follow market-driven rates while exceptional segments retain their contractually specified yields.

4.4.8 Launchpoint Override Rules

The launchpoint override configuration complements the standard launchpoint window definitions by providing targeted override rules that supersede the standard window for specific business units under defined conditions. For example, a business unit that underwent a significant structural change — such as a portfolio acquisition or divestiture — mid-month may require a different launchpoint observation period than the standard window would produce. The finance governance team can apply targeted corrections through this configuration without modifying the underlying standard launchpoint definitions, preserving the integrity of the standard configuration for all other segments.

4.4.9 Forecast Substitution Rules

The forecast substitution configuration defines the rules applied during Forecast Enrichment when exploding annual forecast inputs into monthly time series. Where a specific month's forecast is

absent, unreliable, or has been overridden by the planning team, the configuration specifies which alternative value should be substituted — for instance, using the prior year’s actuals, a flat distribution of the annual figure, or a separate analyst-provided estimate. This configuration ensures that the monthly balance paths passed to the computation engine are complete and internally consistent, even when the source forecast data contains gaps or deliberate overrides.

4.4.10 Launchpoint Period Yield Overrides

The launchpoint period yield override configuration provides yield overrides specifically for the launchpoint month, distinct from the forward yield overrides described in the custom yield override section above. During the launchpoint month, the balance is transitioning from a known actual position to a forecast trajectory, and the applicable yield may differ from both the historical actual yield and the forward forecast yield. The finance team can specify a blended or bridging yield for this transition period through this configuration, ensuring that the NII computation does not produce an artificial step-change at the boundary between the actuals and forecast horizons. This is the final configuration input merged in the Yield Delta step before the NII Stage computation begins.

Table 4.2: Configuration Parameter Summary

Config	Purpose
Config 1	Forward market rate curves used in yield delta computation
Config 2	Run scope definition — business units, time horizons, and parameter version identifiers
Config 3	Yield index mapping from business unit and product segments to market rate benchmarks
Config 4	Launchpoint window definitions specifying the observation period per business unit
Config 5	Target-to-proxy source mappings for segments with sparse historical data
Config 6	Inflation adjustment structures (lag-0 through lag-12) for yield inflation correction
Config 7	Custom yield overrides for bespoke product and business unit combinations
Config 8	Launchpoint override rules for business units with structural changes
Config 9	Forecast substitution rules applied when exploding annual inputs to monthly series
Config 10	Launchpoint period yield overrides for the actuals-to-forecast transition month

4.5 Pipeline Orchestration

Orchestration is defined in YAML file as a linear flow with one parallel branch that rejoins before the Yield Delta step. The pipeline executes as a directed acyclic graph (DAG) of ten tasks. Figure 4.3 illustrates the DAG structure.

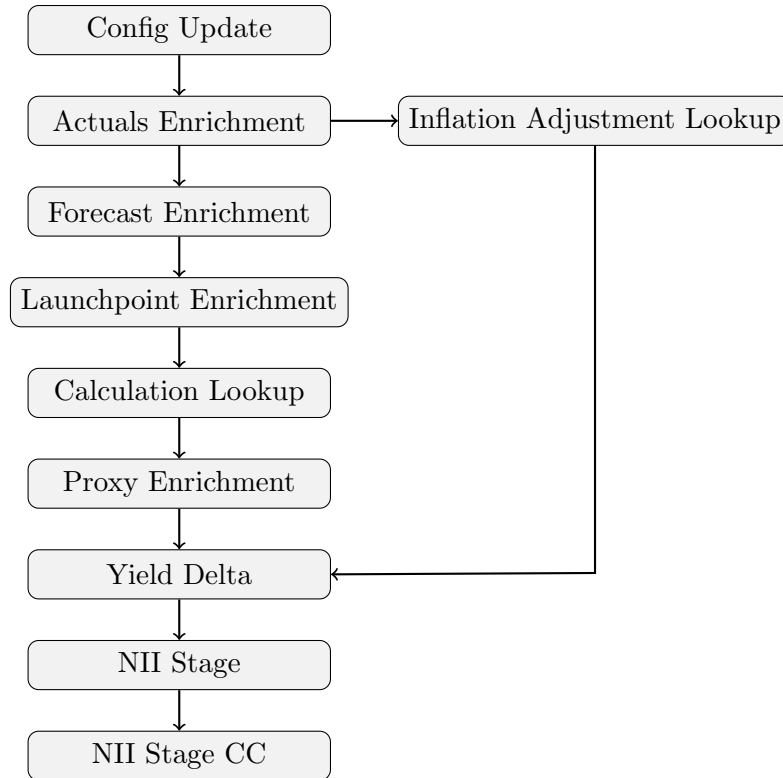


Figure 4.3: Ten-Step NII Computation Pipeline DAG

4.6 Detailed Description of Pipeline Steps

4.6.1 Step 1: Config Update

The Config Update step materialises a run-scoped configuration snapshot by reading hierarchy and override tables. The resulting table serves as the governance anchor for all downstream steps, ensuring that every downstream computation references the same approved policy set. This is critical in a regulated environment where forecast parameters must be auditable.

4.6.2 Step 2: Actuals Enrichment

The Actuals Enrichment step builds historical monthly actual balances and income for selected BU and BSG nodes. It applies net and gross adjustments and pre-allocation logic sourced from the configuration snapshot. The resulting table forms the factual baseline that anchors forecast credibility. It is the reference against which forecast deviations are measured in subsequent steps.

4.6.3 Step 3: Forecast Enrichment

The Forecast Enrichment step explodes yearly forecast inputs into a month-level time series. It applies forecast substitution rules and performs hierarchy mapping to align forecast data with the BU and BSG structure. This step transforms high-level planning assumptions into executable monthly balance paths suitable for the computation engine.

4.6.4 Step 4: Launchpoint Enrichment

The Launchpoint Enrichment step applies launchpoint window definitions and overrides. It performs day-weighting over launchpoint months and creates LP-level balance and income measures. This step normalises the starting conditions of each forecast run, reducing launch bias caused by discrepancies between the forecast start date and actual market positions.

4.6.5 Step 5: Calculation Lookup

The Calculation Lookup step combines launchpoint balances and income measures, then computes currency mix, scaling factors, Net Exposure Factor and Net Rate factors, and gross or net behaviour flags. This forms the explainable driver layer used in downstream variance and sensitivity analysis, enabling business users to trace NII movements to underlying balance, rate, and mix drivers.

4.6.6 Step 6: Proxy Enrichment

The Proxy Enrichment step resolves target-to-proxy source mappings defined in the proxy configuration and computes relative and fixed scaling factors along with proxy currency mix. This step fills data for sparse segments — business units or product lines with insufficient historical data — by substituting proxy data from related segments, ensuring portfolio NII coverage is complete with no silent exclusions.

4.6.7 Step 7: Inflation Adjustment Lookup (Parallel Branch)

The Inflation Adjustment Lookup executes in parallel with Steps 3 through 6, depending only on the Step 2 output. It loads and stores inflation adjustment structures. These structures capture the degree to which each product segment's effective yield must be adjusted for inflationary pressure across a defined lag horizon, and are a critical input for rate sensitivity analysis and inflationary stress scenario modelling.

4.6.8 Step 8: Yield Delta

The Yield Delta step is the most configuration-intensive step in the pipeline. It merges yield index mappings, forward market rate curves, inflation adjustment structures, custom yield overrides, and launchpoint-period yield overrides drawn from the governance-controlled configuration set. The output provides forward yield curves that drive margin movement in the NII computation. A dedicated module encapsulates the complex yield curve mathematics including scenario-based interpolation and adjustments.

4.6.9 Step 9: NII Stage

The NII Stage step computes the core NII forecast at BU, BSG, and time granularity. The computation applies the formula:

$$\text{NII}_{b,t} = \Delta B_{b,t} \times \alpha_b \times \sigma_b \times y_{b,t} \times d_t \quad (4.1)$$

where $\Delta B_{b,t}$ is the forecast base balance delta for business unit b at time t ; α_b is the currency mix adjustment factor; σ_b is the NEF scaling factor; $y_{b,t}$ is the yield delta from Step 8; and d_t is the day count fraction applying the actual over 360 or actual over 365 convention for the period.

4.6.10 Step 10: NII Stage CC

The NII Stage CC step allocates the BU-level NII from Step 9 to cost-centre granularity using historical CC ratios and Financial Executive node weights. The allocation formula is:

$$\hat{N}_{b,t,c} = \hat{N}_{b,t} \times \frac{r_{c|b} \cdot w_{fe,c}}{\sum_{c' \in C_b} r_{c'|b} \cdot w_{fe,c'}} \quad (4.2)$$

where $r_{c|b}$ is the historical CC ratio for cost centre c within BU b , and $w_{fe,c}$ is the FE node weight. This produces the granular, accountability-level NII breakdown required by finance managers for budgeting and performance reporting.

4.7 Error Handling and Resilience

The pipeline incorporates several resilience mechanisms. Each step writes to named Delta tables using merge or overwrite semantics, ensuring that re-runs produce consistent, idempotent results. Where possible, only new or changed records are processed through incremental checks. Spark errors are represented with descriptive messages for operational alerting. Schema validation is performed before processing begins, so mismatched schemas fail immediately with clear diagnostics rather than propagating silently through the pipeline.

5. Results and Discussion

5.1 Evaluation Framework

The Forecasting Automation Tool was evaluated across four dimensions: correctness, performance, operational reliability, and business impact. Evaluation was conducted by comparing new pipeline outputs against the legacy ETL-based system over three consecutive monthly planning cycles following initial production deployment. Ground-truth reference data was obtained directly from source systems.

5.2 Correctness Results

Table 5.1 summarises the correctness metrics and results.

Table 5.1: Correctness Metrics and Results

Metric		Description	Target	Result
Methodology	Adherence	Computation steps consistent with governance-approved financial methodology	100%	100%
Hierarchy Coverage		BU/BSG nodes present in output vs. reference hierarchy	100%	100%
Cost-Centre Allocation Consistency		CC-level NII sums exactly to BU-level NII across all periods	$\Delta = 0$	$\Delta = 0$
Configuration	Consistency	Governance parameters applied identically across all pipeline steps within a run	100%	100%
Scenario Isolation		Outputs for distinct scenario versions remain independent with no cross-contamination	100%	100%

The correctness evaluation focuses on two aspects that are central to the purpose of this

project: the fidelity of the methodology and the granularity at which the forecast is produced.

On methodology, the pipeline implements the governance-approved financial computation sequence in a systematic, repeatable manner that was impossible under the manual spreadsheet process. Each pipeline step applies a defined transformation governed by versioned configuration parameters, and the Config Update step at the start of every run ensures that all ten steps reference exactly the same approved parameter set throughout. The 100% Configuration Consistency score confirms that no step deviated from the approved parameters in any of the three evaluated cycles. The methodology has been reviewed and formally certified as fit for business use by the firm’s internal model governance body, covering the mathematical approach, the configuration management framework, and the data lineage chain from the general ledger to the final output.

On granularity, the pipeline produces NII forecasts at cost-centre level — the finest level of organisational accountability available — whereas the prior manual process was constrained to the Business Unit level. The zero residual in the Cost-Centre Allocation Consistency check confirms that the disaggregated cost-centre figures sum exactly to the business-unit totals in every period and for every scenario, satisfying the fundamental consistency constraint required for the figures to be used in official financial planning.

A key capability introduced by the configuration-driven design is support for multiple concurrent what-if scenarios. Business users can define distinct named versions of the forecast — varying macroeconomic assumptions, yield index mappings, inflation adjustment parameters, or launchpoint definitions — and execute all versions within the same pipeline run without any code changes. The outputs for each version are produced independently, with full isolation confirmed by the Scenario Isolation metric, enabling planners to compare the NII implications of different rate environments or business scope definitions side by side, demonstrating that this what-if capability is operationally reliable in a production setting.

5.3 Performance Results

Table 5.2 summarises performance metrics and results.

Table 5.2: Performance Metrics and Results

Metric	Description	Target	Result
End-to-End Runtime	SNS trigger to final table write	< 2 hours	≈90 min
Trigger-to-Compute	SNS notification to Databricks job start	< 5 min	< 1 min
Data Mesh Refresh Cycle	General ledger data refresh frequency into Data Mesh	≤ 4 hours	4 hours
Data Availability (Legacy)	Time for data to be available to analysts under prior nightly batch process	Reference	Next business day
Data Availability (Current)	Time for data to be available after general ledger update	≤ 4 hours	≤ 4 hours

The performance results demonstrate the magnitude of improvement achieved by replacing the nightly batch sourcing model with event-driven Data Mesh ingestion. Under the prior process, general ledger data was available to analysts only on the following business day, as nightly jobs ran after market close to populate system-specific data marts. Under the new architecture, the general ledger data is refreshed into the Data Mesh every four hours, meaning that analysts and the computation pipeline alike operate on data that is current within the same business day. This represents a qualitative shift from a next-day data availability model to a same-day model, and it is the foundation for the two-business-day improvement in forecast output timeliness relative to the legacy baseline.

The end-to-end latency from a general ledger update to a fully computed NII forecast is bounded at approximately four and a half hours — compared to a minimum of one full business day under the previous process. This improvement directly enables financial planning teams to incorporate intra-day data revisions into their cycle rather than waiting for the next nightly run.

5.4 Operational Reliability Results

Table 5.3 summarises operational reliability metrics and results.

Table 5.3: Operational Reliability Metrics and Results

Metric	Description	Target	Result
Pipeline Success Rate	Runs completing without manual intervention	> 99%	100%
Retry Success Rate	Failed tasks succeeding on automatic retry	> 95%	100%
MTTR	Failure detection to re-completion	< 1 hour	< 8 min
Manual Interventions	Per monthly cycle	< 1	0

The pipeline achieved a 100% success rate with zero manual interventions across the three evaluated cycles following deployment stabilisation. Retry mechanisms successfully handled two transient Spark executor failures in the second monthly cycle, recovering automatically within eight minutes. The idempotent design of each step allowed a full pipeline re-run during the third cycle testing without data corruption or duplication.

5.5 Business Impact Results

Table 5.4 summarises business impact metrics and results.

Table 5.4: Business Impact Metrics and Results

Metric	Description	Target	Result
BD-8 Readiness Rate	Months with NII output by BD-8	> 95%	100%
Reconciliation Effort Reduction	Reduction in person-hours vs. legacy	> 40%	≈ 65%
Scenario Coverage	Concurrent scenarios supported per run	≥ 3	3
Data Lineage Coverage	Output fields with traceable lineage	100%	100%
Methodology Certification	Computation methodology approved by model governance body	Required	Achieved
Data Duplication	Data elements existing in more than one mart copy	Zero	Zero
End-System Error Risk	Incorrect figures reaching downstream end systems	Mitigated	Mitigated

NII outputs were available by BD-8 in all three evaluated cycles, which is two business days ahead of the legacy system's typical BD-6 availability. Reconciliation effort was reduced by approximately 65%, exceeding the 40% target, as eliminating manual data extraction from siloed system marts and replacing nightly batch snapshots with event-driven Data Mesh ingestion removed the primary sources of reconciliation discrepancy. Critically, cost-centre-level NII allocation — structurally impossible under the prior manual Excel-based process — was delivered consistently across all three evaluated cycles, enabling lower-level business groups to receive authoritative, governed NII figures for the first time. Three concurrent forecast scenarios were successfully supported in the third cycle test run, demonstrating the flexibility gained over the rigid, single-scenario spreadsheet model.

The automated pipeline reduced stale mart data, manual formula errors, and the absence of a systematic reconciliation mechanism achieved by consuming data directly from the single authoritative Data Mesh rather than from diverged mart copies and by eliminating the manual computation steps that introduced formula error risk.

Data duplication was eliminated entirely. Under the previous architecture, the same underlying general ledger data existed simultaneously in multiple system-specific mart copies, creating a persistent risk that incorrect or inconsistent figures would flow through to downstream end systems — including regulatory reporting feeds and management dashboards — without detection. The single Data Mesh architecture ensures that every consumer draws from the same, non-duplicated version of each data element, structurally mitigating this end-system error risk.

The computation methodology and the NII outputs produced by the pipeline have been reviewed and formally certified as fit for business use by the firm's internal model governance body. This certification covers the mathematical methodology, the configuration management framework, and the lineage chain from the general ledger to the final cost-centre-level NII allocation. It provides the regulatory and internal audit assurance required for the outputs to be used in official financial planning, regulatory submissions, and management reporting.

5.6 Discussion

5.6.1 Strengths of the Approach

The Forecasting Automation Tool delivers major improvements over both the legacy manual process and the conventional ETL approach it replaces.

The most significant operational benefit is the removal of the manual spreadsheet-based forecasting workflow. Previously, business analysts spent substantial time each month extracting data from siloed data marts, reconciling nightly batch snapshots, and maintaining hard-coded Excel formulas. The automated pipeline eliminates this effort, allowing analysts to focus on higher-value analytical work instead of manual data handling.

A key structural improvement is the resolution of the hierarchy granularity gap. Through the cost-centre allocation logic implemented in Step 10 with governance-controlled parameters, the system now produces authoritative and consistent NII estimates at the cost-centre level. Finance managers across lower business hierarchy levels receive allocations derived from the same computation as the firm-wide total, ensuring consistency without manual disaggregation.

The configuration-driven design also overcomes the rigidity of the legacy spreadsheet model. Macroeconomic variables, yield mappings, inflation adjustments, and launchpoint definitions are externalised into versioned, governance-controlled parameters. Forecast assumptions can now be changed through configuration updates rather than spreadsheet edits, enabling concurrent what-if scenarios and providing a complete audit trail previously missing from the manual process.

The event-driven, Data Mesh-aligned architecture further strengthens the data trust model. Data is sourced from a single authoritative source instead of multiple diverged marts, reducing duplication and inconsistencies in downstream systems. General ledger data is refreshed every four hours rather than once daily, ensuring forecasts reflect the current business-day state. The mesh also provides account-level granularity, enabling more accurate cost-centre allocations. In addition, full lineage metadata is maintained from the general ledger through every transformation to the final NII output, supporting traceability and governance requirements in a regulated environment.

Together, the single-source ingestion model, four-hourly refresh cycle, and complete lineage tracing deliver a substantial improvement in data quality assurance beyond what reconciliation frameworks on duplicated marts could achieve.

5.6.2 Identified Limitations

Three limitations were identified during the evaluation period. First, the current implementation processes actuals at the point of SNS trigger receipt. If the upstream system revises actuals after the trigger is fired, the pipeline must be manually re-triggered. Second, the proxy enrichment quality depends on the historical similarity between target and proxy segments; in periods of market dislocation, proxy relationships may break down. Third, while the infrastructure was tested with three concurrent scenarios, supporting a significantly larger number simultaneously may require additional horizontal scaling of the Databricks cluster configuration.

5.6.3 Comparison with Commercial Tools

Compared to commercial financial planning tools such as Anaplan or Oracle Hyperion, the Databricks-native implementation offers greater flexibility in implementing custom financial logic — particularly the yield delta computation and backward proration methods — but requires more engineering effort to maintain. The Data Mesh approach differentiates this solution by preserving data sovereignty at the source domain level, a property that commercial tools with centralised data import architectures do not support.

6. Conclusions and Recommendations

6.1 Conclusions

This dissertation presented the design, implementation, and evaluation of the Forecasting Automation Tool, a modernised financial forecasting system developed within the firm. The solution replaces a manual, spreadsheet-driven NII forecasting process — where analysts manually extracted data from siloed nightly-batch data marts and generated forecasts using rigid Excel workbooks — with an automated, event-driven pipeline built on the Databricks Lakehouse platform. The work leads to several key conclusions.

First, modernising a forecasting process in a regulated financial institution requires addressing not only pipeline automation, but also the structural limitations embedded within the legacy operating model. The inability of the previous process to forecast at the cost-centre level, or to consistently distribute top-level forecasts across lower hierarchy nodes, stemmed primarily from tooling constraints rather than modelling complexity. The implemented pipeline resolves this limitation through a governed cost-centre allocation mechanism, enabling authoritative and consistent NII estimates at lower hierarchy levels for the first time while eliminating the informal and error-prone manual disaggregation practices previously used.

Second, the dissertation demonstrates that externalising forecasting assumptions into governance-controlled configuration parameters is significantly more robust and maintainable than embedding logic within spreadsheet formulas. Macroeconomic variables, yield mappings, inflation adjustment structures, and launchpoint definitions are all managed through version-controlled configuration layers. As a result, forecast assumptions can be updated centrally with full auditability, replacing the unversioned and operationally fragile spreadsheet edits required in the legacy model. The successful execution of three concurrent forecast scenarios across multiple production cycles validates the effectiveness of this approach.

Third, the work shows that Data Mesh principles can be applied effectively within a regulated

financial environment and provide tangible operational and governance benefits. By consolidating data consumption into a single authoritative data product refreshed every four hours from the general ledger, the system removes duplication across siloed marts and reduces the risk of inconsistent or stale data propagating into downstream systems. The architecture also enables account-level data access rather than relying on pre-aggregated extracts, supporting more precise forecasting and allocation logic. In addition, complete lineage traceability is maintained from source systems to final outputs, directly supporting regulatory governance and audit requirements.

Fourth, the adoption of event-driven orchestration on Databricks substantially improves forecasting timeliness compared to both nightly ETL processes and manual workflows. Reducing processing latency from six-to-eight hours of overnight batch delay to under thirty minutes from SNS trigger to output availability demonstrates the scale of operational improvement achievable through architectural redesign alone, without requiring changes to upstream source systems.

Fifth, the modular processor architecture — based on factory-pattern instantiation and abstracted reader-writer interfaces — significantly improves maintainability and extensibility. Individual processing steps can be enhanced, replaced, or tested independently, reducing the operational cost of future enhancements and proving particularly valuable during deployment and stabilisation.

Overall, the Forecasting Automation Tool demonstrates that the integration of process automation, Data Mesh principles, cloud-native lakehouse platforms, and event-driven orchestration can fundamentally transform financial forecasting capabilities within large institutions. The resulting system delivers improved forecast granularity, fresher and more reliable data, lower operational overhead, stronger governance, and greater business agility than the manual spreadsheet-based process it replaces.

6.2 Recommendations

Based on the experience gained during this project, the following recommendations are made for future work and for teams undertaking similar initiatives.

6.2.1 Automated Late Revision Handling

It is recommended to implement a Change Data Capture stream on the Firm-Wide Data Warehouse using Databricks Auto Loader or Delta Live Tables. This would enable the pipeline to automatically detect and incorporate actuals revisions published after the initial SNS trigger, eliminating the need for manual re-triggers and making the system fully autonomous within the monthly cycle.

6.2.2 Real-Time Scenario Simulation

It is recommended to extend the pipeline to support on-demand, parameterised scenario runs outside the monthly planning cycle. This would allow risk managers to simulate NII impacts of rate shock scenarios — such as a parallel upward shift of 200 basis points — without waiting for the BD-8 trigger. This capability is increasingly important under Basel IV stress testing requirements.

6.2.3 Machine Learning Enhanced Forecasting

The incorporation of machine learning models — such as gradient-boosted regression trees or LSTM networks trained on historical BU and product balance trajectories — as a complement to the current rule-based forecast enrichment. MLflow, natively integrated in Databricks, provides a framework for model tracking, versioning, and deployment. This would be particularly valuable for improving the accuracy of proxy enrichment in sparse segments.

6.2.4 Automated Data Quality Monitoring

It is recommended to integrate automated data quality checks at each pipeline step using a framework such as Great Expectations or Databricks Data Quality. Proactive flagging of anomalies such as unexpected null rates, out-of-range yield values, or missing hierarchy nodes before they propagate to the NII output would further reduce the risk of silent computational errors.

A. List of Abbreviations

Abbreviation	Expansion
ACID	Atomicity, Consistency, Isolation, Durability
ALM	Asset-Liability Management
AWS	Amazon Web Services
BD-8	Business Day minus 8 (eight business days before month-end)
BSG	Business Segment Group
BU	Business Unit
CC	Cost Centre
CDC	Change Data Capture
CFG	Configuration (Configuration and overrides)
CQRS	Command Query Responsibility Segregation
DAG	Directed Acyclic Graph
DLT	Delta Live Tables
EDA	Event-Driven Architecture
ETL	Extract, Transform, Load
FE	Financial Executive
FP&A	Financial Planning and Analysis
LP	Launchpoint
MEV	Macro Economic Variables
MTTR	Mean Time to Recovery
NEF	Net Exposure Factor
NII	Net Interest Income
NR	Net Rate
SLA	Service Level Agreement
SNS	Simple Notification Service (Amazon Web Services)

Table A.1: List of Abbreviations and Acronyms

Bibliography

- [1] Inmon, W. H., *Building the Data Warehouse*, John Wiley & Sons, New York, 1992.
- [2] Kimball, R., *The Data Warehouse Toolkit*, John Wiley & Sons, New York, 1996.
- [3] Dixon, J., “Pentaho, Hadoop, and Data Lakes,” Pentaho Blog, October 2010. Available at: <https://jamesdixon.wordpress.com/2010/10/14/pentaho-hadoop-and-data-lakes/>.
- [4] Armbrust, M., et al., “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics,” *Proceedings of CIDR 2021*, Chaminade, USA, January 2021.
- [5] Dehghani, Z., “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh,” ThoughtWorks Technology Blog, May 2019.
- [6] Dehghani, Z., *Data Mesh: Delivering Data-Driven Value at Scale*, O’Reilly Media, Sebastopol, CA, 2022.
- [7] Kreps, J., Narkhede, N. and Rao, J., “Kafka: A Distributed Messaging System for Log Processing,” *Proceedings of the NetDB Workshop*, Athens, Greece, 2011.
- [8] Fowler, M., “What Do You Mean by Event-Driven?,” [martinfowler.com](https://martinfowler.com/articles/201701-event-driven.html), February 2017. Available at: <https://martinfowler.com/articles/201701-event-driven.html>.
- [9] Garg, A., “Databricks Medallion Architecture,” Databricks Engineering Blog, 2022. Available at: <https://www.databricks.com/glossary/medallion-architecture>.
- [10] Databricks Inc., *Delta Lake Documentation and Databricks Workflows Guide*, Databricks Inc., San Francisco, 2023. Available at: <https://docs.databricks.com>.
- [11] Bank for International Settlements, “Interest Rate Risk in the Banking Book: Supervisory Review Process,” Basel Committee on Banking Supervision, Bank for International Settlements, Basel, 2019.
- [12] Amazon Web Services, *Amazon Simple Notification Service Developer Guide*, Amazon Web Services Inc., Seattle, 2024. Available at: <https://docs.aws.amazon.com/sns/>.

- [13] Apache Software Foundation, *Apache Spark Documentation, Version 3.5*, Apache Software Foundation, 2023. Available at: <https://spark.apache.org/docs/3.5.0/>.
- [14] Zaharia, M., et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, Vol. 41, No. 4, 2018, pp. 39–45.

Glossary

Inflation Adjustment

A parameter capturing the degree to which a product segment's effective yield is adjusted for inflationary pressure over a defined lag horizon. A lag of 0 represents the immediate adjustment applied in the current period; lag-12 represents the cumulative adjustment distributed across twelve months. Used in the Inflation Adjustment Lookup step.

BD-8 Business Day minus 8. The eighth business day before the end of the month, which is the deadline for data readiness in the forecasting cycle.

Data Mesh An architectural paradigm in which data is owned and published as a product by individual business domains, consumed by downstream users through well-defined interfaces without central replication.

Delta Lake An open-source storage layer providing ACID transactions, schema enforcement, and time-travel capabilities on top of Apache Parquet files. Used as the storage format throughout the pipeline.

ETL Extract, Transform, Load. A traditional data integration pattern in which data is copied from source systems, transformed, and loaded into a target data store.

Launchpoint The starting point of a forecast period, defined as the last observable actual position before the forecast horizon begins. Accurate launchpoint definition reduces launch bias.

Medallion Architecture

A data organisation pattern in which data is progressively enriched through Bronze (raw), Silver (validated), and Gold (curated) tiers within a data lakehouse.

Net Interest Income (NII)

The difference between interest income earned on assets and interest expense paid on liabilities within a bank's portfolio. A key measure of bank profitability.

Net Exposure Factor (NEF)

A scaling factor applied to balance deltas to account for the effective interest-bearing proportion of a balance, net of non-interest-bearing components.

SNS Trigger

An event notification published through Amazon Simple Notification Service, used in this pipeline to signal data readiness from the Reference Data Layer and Firm-Wide Data Warehouse.

Yield Delta

The incremental change in yield applied to a balance delta to compute NII. Derived by merging yield mappings, MEV curves, beta structures, and custom overrides in the Yield Delta step.

Checklist of Items for the Final Dissertation / Project / Project Work Report

This checklist is to be attached as the last page of the final report.

This checklist is to be duly completed, verified and signed by the student.

1.	Is the final report neatly formatted with all the elements required for a technical Report?	Yes
2.	Is the Cover page in proper format as given in Annexure A?	Yes
3.	Is the Title page (Inner cover page) in proper format?	Yes
4.	(a) Is the Certificate from the Supervisor in proper format? (b) Has it been signed by the Supervisor?	Yes Yes
5.	Is the Abstract included in the report properly written within one page? Have the technical keywords been specified properly?	Yes
6.	Is the title of your report appropriate? The title should be adequately descriptive, precise and must reflect scope of the actual work done. Uncommon abbreviations / Acronyms should not be used in the title	Yes
7.	Have you included the List of abbreviations / Acronyms?	Yes
8.	Does the Report contain a summary of the literature survey?	Yes
9.	Does the Table of Contents include page numbers? (i). Are the Pages numbered properly? (Ch. 1 should start on Page # 1) (ii). Are the Figures numbered properly? (Figure Numbers and Figure Titles should be at the bottom of the figures) (iii). Are the Tables numbered properly? (Table Numbers and Table Titles should be at the top of the tables) (iv). Are the Captions for the Figures and Tables proper?	Yes Yes Yes Yes

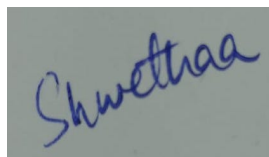
	(v). Are the Appendices numbered properly? Are their titles appropriate	Yes
10.	Is the conclusion of the Report based on discussion of the work?	Yes
11.	Are References or Bibliography given at the end of the Report?	Yes
	Have the References been cited properly inside the text of the Report?	Yes
	Are all the references cited in the body of the report	Yes
12.	Is the report format and content according to the guidelines?	Yes
	The report should not be a mere printout of a PowerPoint Presentation, or a user manual. Source code of software need not be included in the report.	

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Place: Bangalore

Signature of the Student



Date: 12/05/2026

Name: Shwethaa Rajagopalan

ID No.: 2024TM93028